Testbericht 3 – UI Phase B: Griffe, Snap, Parameterleiste

Bearbeiten in der Ansicht · 23. Oktober 2026 · Commit af6edd6

Phase B (Playwright)	Phase A (Regression)	Vitest	Rust (geänderte Crates)	Ziehen → Szene grob (Würfel / Lochplatte)
14 / 14 bestanden	**22 / 22**	**128 / 128**	**33 / 33**	**36 ms / 371 ms**

1 Umfang

Phase B ergänzt die Browser-UI um das Bearbeiten in der 3D-Ansicht. Der Code bleibt die einzige Quelle: Griffe ändern Literale oder schreiben Rasterreferenzen, die Parameterleiste ändert nur die Vorschau.

- **Griffe** der Zeile unter dem Cursor, nur bei Literal oder Variable mit Literal; berechnete Werte werden als gesperrter Griff (grauer Ring) gezeigt. Arten: cube-/square-Ecke, box-/rect-Eckpunkte, Radius (sphere, circle, cylinder, torus), Höhe (cylinder, linear_extrude), cylinder zwischen zwei Punkten, translate-Ziel (Pfeil alt → neu, Griff am Ziel), Drehring pro Achse (rotate), Skalierung an der Hüllenecke, Rasterebenen mit Literal (Griff auf einer Kante der Rasterbox, Ziehen entlang der Achse), Punktvariablen, polygon-/spline-Punkte, Zonenmittelpunkt über translate. Farben: Literal orange, Standardwert eines Modulparameters violett, reine Referenz türkis, gesperrt grau.
- **Ziehen:** `DragStart` → `Drag` (höchstens eine Nachricht in Flug, letzte Position gewinnt) → `Edits` → Editor wendet sie als Live-Änderung an und sendet den Quelltext sofort; die Engine tesselliert grob, nach `DragEnd` fein. Ein Ziehvorgang ist ein Undo-Schritt (Live-Änderungen ohne Verlauf, am Ende als ein isoliertes Ereignis). Esc bricht ab.
- **Snappen** im Bildschirmraum (Fangradius einstellbar, Standard 14 px) auf Rasterpunkte des Moduls und sichtbare Anker (`SnapTargets`), Ziel hervorgehoben und beschriftet; geschrieben wird die Referenz. 1D-Griffe (Höhe, Radius, Ebene) snappen auf Rasterebenen derselben Achse und schreiben `g.name` (neu in cad-eval). **Alt:** kein Snap, Zahlen gerundet auf die Schrittweite (Einstellungen, Standard 1 mm / 15°).
- **Parameterleiste** neben der Ansicht im Modulkontext: Zahl (Eingabefeld + Schieberegler), Bool (Schalter), Vektor (drei Felder); Reset pro Parameter, Markierung geänderter Werte, „Alle zurücksetzen“, Werte pro Modul bis Sitzungsende, „Als Standard übernehmen“ (`SetDefault`). Vorschau über `SetContext { overrides }`.
- **Inline-Regler** für Zahl- und Bool-Literale von Zuweisungen am Zeilenende (Schieberegler, Schalter); ein Schieberegler-Zug ist ein Undo-Schritt.
- **Strg+Klick** auf einen Raster- oder Ankerausdruck springt zur Deklaration.

Änderungen an Rust-Crates: `cad-eval` (gesperrte Griffe als `locked_handles`, `SnapRef::Plane` mit Prüfung, dass der Griff nach dem Schreiben exakt auf der Ebene liegt und die Ebene an der Stelle sichtbar ist), `cad-session` (`Drag` antwortet immer, `SetDragOptions`, Szene mit `locked_handles`), Protokoll-Typen neu erzeugt.

2 Testumgebung

Betriebssystem	Linux 6.18.44-fc-v140 (x64)
CPU	Intel(R) Xeon(R) Processor @ 2.80GHz · 4 Kerne

Node.js	v22.22.0
Browser	Chromium 141.0.7390.37 (headless, Playwright 1.56.1)
Grafik	WebGL über SwiftShader (Software-Rendering, keine GPU)
Fenster	1440 × 900 px, Gerätepixelverhältnis 1
Build	Vite 8.3.5 Produktions-Build, Vorschau-Server (vite preview)
Bibliotheken	React 19.3.1, three.js 0.186.1, CodeMirror 6.43.15, Zustand 5.0.16, Vitest 5.0.5
Engine	cad_wasm_bg.wasm 1.17 MiB, smlib.wasm 4.85 MiB
Quelltext	Branch worktree-agent-a590210bcbc87a815, Commit af6edd6

Die End-to-End-Tests laufen gegen den Produktions-Build mit echten Mausereignissen auf dem Canvas (Playwright `mouse.down/move/up`, ein Schritt pro Animationsframe). Gewartet wird auf Szenen-Versionen, den Zieh-Zustand und den Editortext über den Testhaken `window.__cad`, nie auf feste Zeiten. Der Haken liefert nur Bildschirmpositionen (Griffe, Rasterpunkte, Punkte auf Ring oder Achse).

3 Szenarien (Playwright, web/e2e/ui-b.spec.ts)

ID	Szenario	Ergebnis	Dauer	Prüfung
B01	cube-Ecke ziehen: Literal live, Szene folgt, ein Undo	**bestanden**	7.6 s	Griff an der Ecke gegenüber dem Ursprung mit der Maus gezogen: das Literal änderte sich live (6 Zwischenstände mit grober Tessellierung), die Szene folgte. Ein Strg+Z stellte „cube([20, 20, 20]);“ wieder her, Strg+Y (Wiederholen) wieder „cube([22, 23, 22]);“.
B02	translate-Griff auf Rasterpunkt loslassen schreibt g[...]	**bestanden**	13.7 s	translate-Griff (Ziel des Pfeils) zum Schnittpunkt rechts/hinten/deckel gezogen: Ziel im Fangradius hervorgehoben und beschriftet, beim Loslassen steht die Referenz g[rechts, hinten, deckel] im Code (keine Koordinaten).
B03	Snap auf Anker schreibt k.ecke	**bestanden**	13.1 s	Griff des translate-Ziels auf die Raute des Ankers ecke der benannten Instanz k gezogen: Beschriftung „k.ecke“, beim Loslassen wird k.ecke geschrieben.
B04	Alt-Ziehen: frei, Zahlen auf die Schrittweite gerundet	**bestanden**	17.2 s	Mit gedrückter Alt-Taste nahe an einen Rasterpunkt gezogen: kein Snap, es werden Zahlen geschrieben, gerundet auf 1 mm (translate([39, 31, 1]) cube(4);). Danach Schrittweite in den Einstellungen auf 0,5 mm gesetzt: translate([37, 28.5, -0.5]) cube(4);.
B05	Rasterebene mit Literal ziehen	**bestanden**	5.9 s	Cursor im Rasterblock: alle Ebenen mit Literal als durchgezogene Linie mit Griff. Der Griff der Ebene rechts wurde entlang der x-Achse gezogen; das Literal folgt, Linie und Rasterpunkte wandern mit.
B06	Höhe snappt auf Rasterebene derselben Achse (g.deckel)	**bestanden**	6.5 s	Höhengriff des Zylinders an die Rasterebene deckel gezogen: Ebenen derselben Achse sind Snap-Ziele, geschrieben wird g.deckel (neu in cad-eval: SnapRef::Plane).
B07	rotate-Ring rastet in 15°	**bestanden**	7.8 s	Drehringe pro Achse; der Griff am z-Ring wurde von 10° bis 52° gezogen. Geschrieben werden nur Vielfache von 15° (Schrittweite für Winkel), am Ende 45°.
B08	gesperrter Griff bei berechnetem Wert	**bestanden**	2.9 s	Berechneter Wert [a + 1, 0, 0] * 4: der Griff wird angezeigt (grauer Ring), ist aber gesperrt. Ziehen ändert nichts, die Statuszeile meldet „berechneter Wert – gesperrt“.

B09	Variable mit Literal: Definition ändert sich, alle Verwendungen bewegen sich	**bestanden**	8.8 s	translate(p) in Zeile 2 gezogen: geändert wird die Definition von p in Zeile 1, Würfel und Kugel (Zeilen 2 und 3) verschieben sich um denselben Betrag.
B10	Zonenmittelpunkt über translate snappt auf Raster	**bestanden**	12.3 s	Cursor auf der Zone (Kugel) im fillet-Block: Griff am Zonenmittelpunkt (translate), auf die Ecke rechts/hinten/deckel gezogen. Die Zone wandert, die ausgewählten Kanten folgen.
B11	Parameterleiste: Vorschau, Reset, Als Standard übernehmen	**bestanden**	5.0 s	Cursor in module gehaeuse: Leiste mit w, d, h, t (Eingabefeld + Schieberegler, Reset ↺). w = 60 änderte nur die Vorschau (Code unverändert, Zeile markiert), Reset stellte 40 her; h per Schieberegler auf 31. d = 50 blieb beim Verlassen und Wiederbetreten des Moduls erhalten; „Als Standard übernehmen" schrieb d = 50 in die Moduldefinition.
B12	Inline-Schieberegler und -Schalter	**bestanden**	3.7 s	Zahl-Literal einer Zuweisung: kleiner Schieberegler am Zeilenende, mit der Maus von 3 auf 7 gezogen (Code und Szene folgen live, ein Undo-Schritt). Bool-Literal: Schalter, Klick schreibt false; die Szene wechselt auf den else-Zweig.
B13	Strg+Klick auf Rasterausdruck springt zur Rasterdeklaration	**bestanden**	1.5 s	Strg+Klick auf g[links, vorne, boden] (Zeile 11) setzt den Cursor auf die Deklaration grid g (Zeile 4); ein normaler Klick setzt weiter nur den Cursor.
B14	Messwerte: Latenz Ziehen → Szene (grob/fein), fps beim Ziehen	**bestanden**	25.5 s	lochplatte (Differenz mit fillet = 1, vier Bohrungen): Ebene oben = t (Standardwert, andere Farbe) 78-mal bewegt, ein Schritt pro Frame. Ziehen → Szene grob: Median 371 ms, p95 738 ms (15 Szenen); Loslassen → feine Szene 202 ms; 21 fps während des Ziehens, 76 Drag-Nachrichten, 0 übersprungen.

Regression: die 22 Szenarien aus Phase A (web/e2e/ui.spec.ts) liefen im selben Lauf, 22 bestanden.

4 Unit-Tests

Vitest-Datei	bestanden	Fehlgeschlagen
web/src/editor/decorations.test.ts	7 / 7	–
web/src/editor/editGroup.test.ts neu	7 / 7	–
web/src/editor/lang.test.ts	42 / 42	–
web/src/logic/context.test.ts	5 / 5	–
web/src/logic/display.test.ts	15 / 15	–
web/src/logic/drag.test.ts neu	9 / 9	–
web/src/logic/format.test.ts neu	5 / 5	–
web/src/logic/grids.test.ts	5 / 5	–
web/src/logic/handles.test.ts neu	12 / 12	–
web/src/logic/params.test.ts neu	7 / 7	–
web/src/logic/positions.test.ts	5 / 5	–
web/src/logic/sceneIndex.test.ts	9 / 9	–

Neu: Snap-Suche im Bildschirmraum, Strahl → Wert (Ebene, Achse, Ring), Vorschau-Rundung, „höchstens ein Drag in Flug", Rundung und Zahlendarstellung, Undo-Gruppierung mit dem echten CodeMirror-Verlauf, Abbildung der Engine-Edits, Zustand der Parameterleiste, Inline-Regler. Fixture `griffe.scad` aus der echten Engine.

Rust-Testprogramm	bestanden	neue Tests
tests/handles.rs	24 / 24	`computed_values_have_locked_handles`, `plane_snap_respects_visibility`, `one_d_handles_snap_to_grid_planes`, `zone_center_handle_through_translate`
tests/protocol_ts.rs	1 / 1	–
tests/session.rs	8 / 8	`drag_always_answers_and_plane_snap`

5 Messwerte

Größe	Wert	Messung
Ziehen → Szene, grob (Median)	371 ms	Drag-Nachricht bis die Szene des daraus geschriebenen Quelltexts gezeichnet ist, grobe Tessellierung (10 × Toleranz); 15 Szenen
Ziehen → Szene, grob (p95 / max)	738 ms / 738 ms	95-%-Quantil und Maximum derselben Messung
Loslassen → feine Szene	202 ms	pointerup bis die Szene mit feinem Wurzel-Mesh gezeichnet ist (DragEnd, Undo-Gruppe schließen, Neuauswertung aus dem Cache)
Bildrate beim Ziehen	21 fps	requestAnimationFrame-Zähler während des Ziehens (19.9 s); längste Lücke 349 ms
Mausbewegungen / Drag-Nachrichten	78 / 76	höchstens eine Nachricht in Flug, 0 Zwischenstände übersprungen (letzte Position gewinnt)
Leichtes Modell: Ziehen → Szene grob (Median / p95)	36 ms / 172 ms	B01, cube([20, 20, 20]): 6 Szenen; Aufwand von UI, Worker und Protokoll ohne nennenswerte Kernel-Arbeit
Leichtes Modell: Loslassen → fein / fps	23 ms / 35 fps	längste Frame-Lücke 412 ms, 15 Drag-Nachrichten
Größte Bilddatei im Bericht	57 KiB	JPEG, Qualität 82

Oberer Teil: spec/lochplatte.scad, Modul lochplatte, Ebene oben = t (Standardwert). Jede Änderung von t wertet die Differenz mit Verrundung und vier Bohrungen vollständig neu aus (kein Cache-Treffer); die Latenz ist hier von der Kernel-Zeit bestimmt (SetSource → vollständige Szene Median 174 ms). Gleiche gerundete Werte erzeugen keine Änderung; jede Änderung wartet auf ihre Szene, bevor die nächste Position gesendet wird (sonst würde jeder neue Quelltext den laufenden Kernel-Auftrag abbrechen). Die Bildrate ist durch das Software-Rendering (SwiftShader) begrenzt; Phase A maß beim Drehen 14 fps.

6 Bekannte Grenzen und Abweichungen

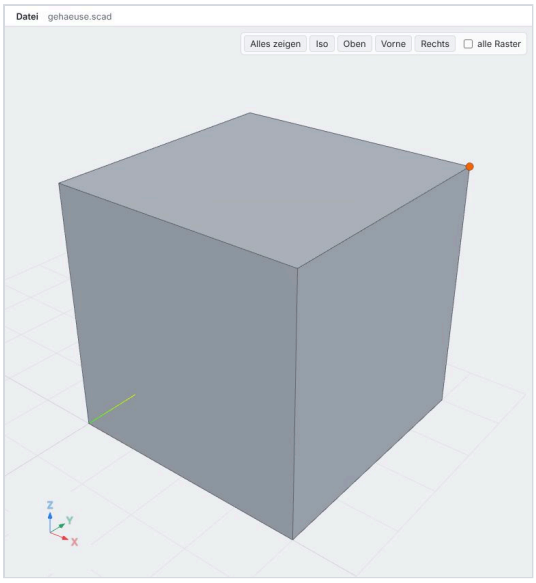
- **Sprung zur Rasterdeklaration mit Strg+Klick** statt einfachem Klick: ein einfacher Klick muss weiter den Cursor setzen, sonst wären Rasterausdrücke nicht editierbar. Referenzen eines Moduls kennt die UI erst im Modulkontext (Cursor im Modul).
- **Ohne Alt** wird gesnappt, wenn ein Ziel im Fangradius liegt, sonst werden Zahlen geschrieben (gerundet). Ein Griff, der nur aus Referenzen besteht (z. B. `box(g[…], g[…])`), wird ohne Alt nie zu Zahlen; er wechselt nur zwischen Zielen.
- **Snap von 1D-Griffen** nur auf Ebenen, auf denen der Griff nach dem Schreiben von `g.name` exakt landet (gleicher Ursprung und Maßstab entlang der Achse). Unter einem verschobenen Teilbaum oder bei `center = true`/`d =` (Faktor 2) gibt es daher keine Ebenen-Ziele; geschrieben würde sonst ein Ausdruck wie `g.deckel - 5`, der den Griff sperrt.

- Punktgriffe werden in der Ebene durch den Griff senkrecht zur Blickrichtung gezogen (2D-Punkte in ihrer xy-Ebene); entlang der Blickrichtung ändert sich nichts. Für eine bestimmte Achse hilft eine Standardansicht.
- Undo-Gruppe: am Ende des Ziehens wird die Gesamtänderung einmal zurückgenommen und als ein Verlaufseintrag wieder angewendet; dabei entsteht eine zusätzliche Neuauswertung aus dem Cache.
- Inline-Regler erscheinen nur bei Zuweisungen mit Zahl- oder Bool-Literal (Spec, Werttypen); der Bereich richtet sich nach dem Wert beim Start des Ziehens (0 … 2 × Wert, mindestens 10).
- Parameter ohne Standardwert werden beim Reset auf 0 / false / [0, 0, 0] gesetzt (Spec-Tabelle); Zeichenketten und Listen sind nur lesbar.
- Die Messwerte stammen aus headless Chromium mit SwiftShader.

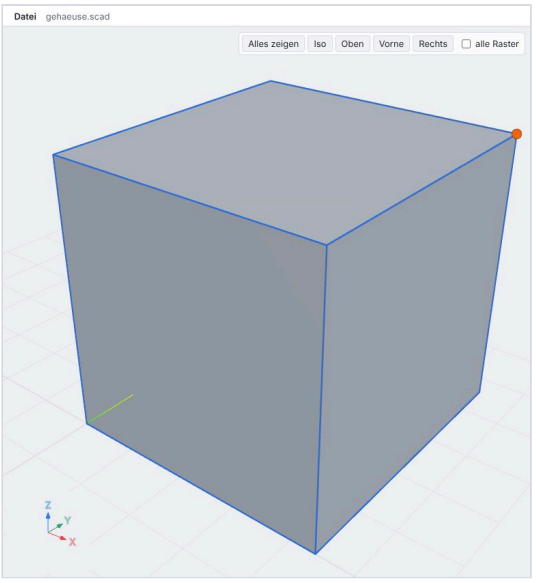
7 Screenshots und Codeausschnitte

B01 cube-Ecke ziehen

VORHER



NACHHER



CODE VORHER

```
cube([20, 20, 20]);
```

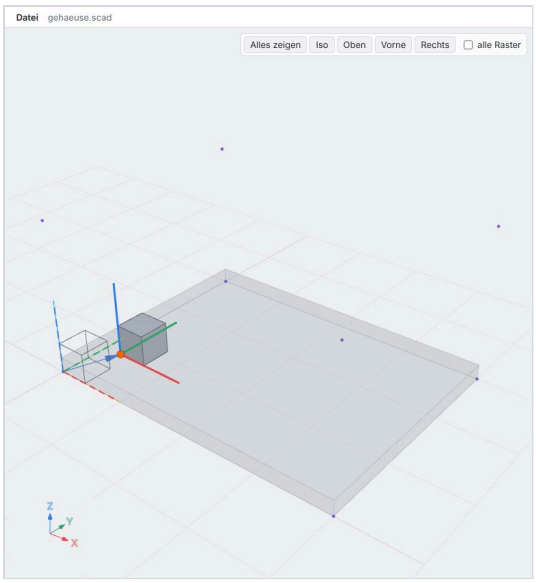
CODE NACHHER

```
cube([22, 23, 22]);
```

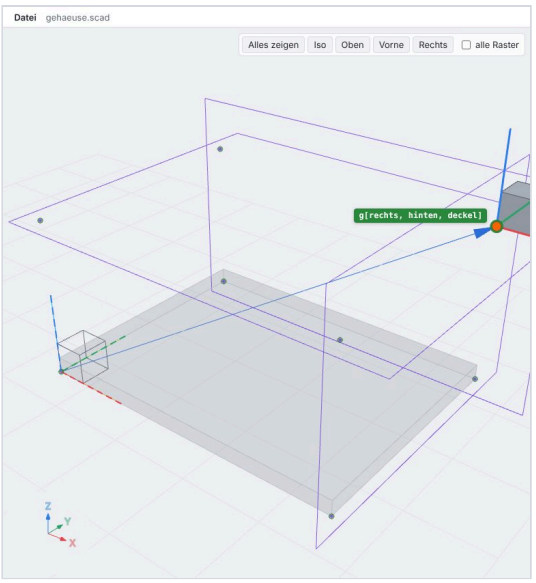
Griff an der Ecke gegenüber dem Ursprung mit der Maus gezogen: das Literal änderte sich live (6 Zwischenstände mit grober Tessellierung), die Szene folgte. Ein Strg+Z stellte „cube([20, 20, 20]);“ wieder her, Strg+Y (Wiederholen) wieder „cube([22, 23, 22]);“.

B02 Snap auf Rasterpunkt

VORHER



WÄHREND



CODE VORHER

```
translate([5, 5, 2]) cube(4);
```

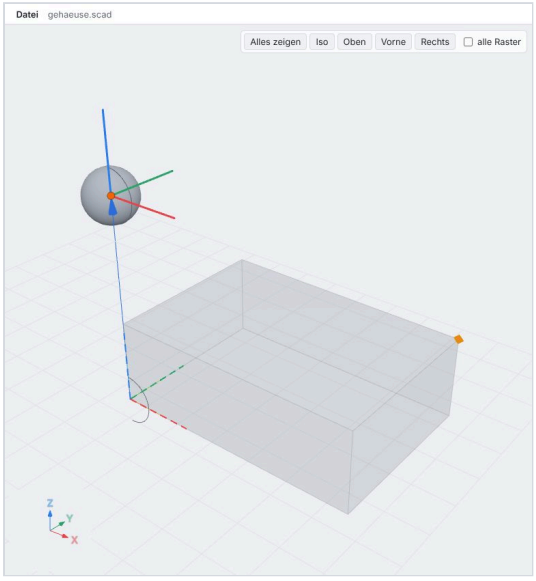
CODE NACHHER

```
translate(g[rechts, hinten, deckel]) cube(4);
```

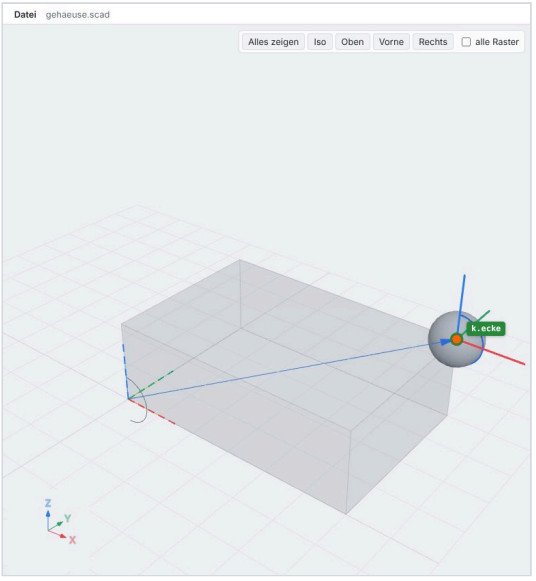
translate-Griff (Ziel des Pfeils) zum Schnittpunkt rechts/hinten/deckel gezogen: Ziel im Fangradius hervorgehoben und beschriftet, beim Loslassen steht die Referenz g[rechts, hinten, deckel] im Code (keine Koordinaten).

B03 Snap auf Anker

VORHER



WÄHREND



CODE VORHER

```
translate([0, 0, 25]) sphere(3);
```

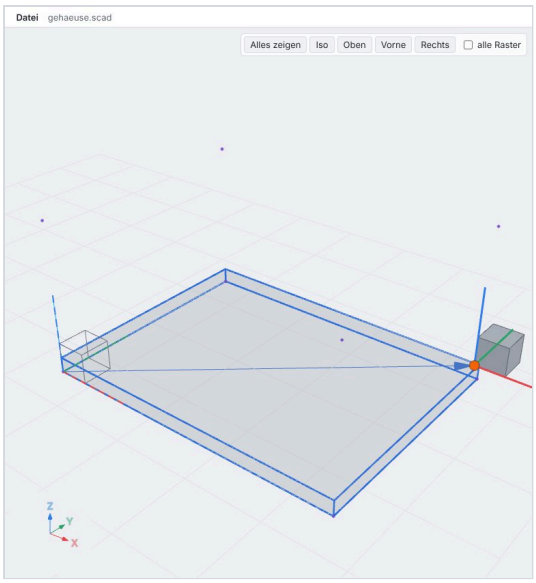
CODE NACHHER

```
translate(k.ecke) sphere(3);
```

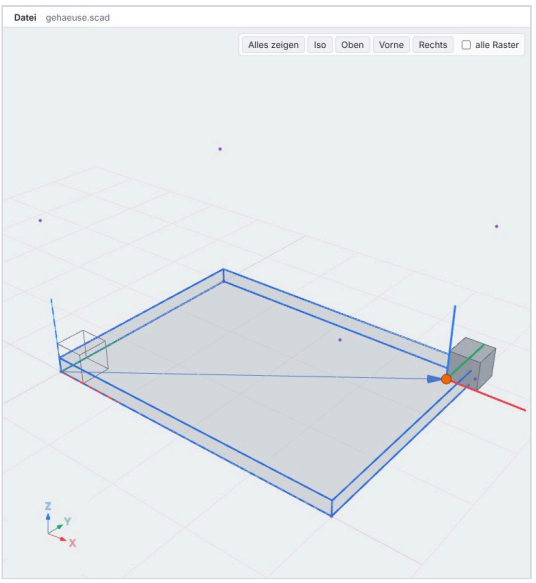
Griff des translate-Ziels auf die Raute des Ankers ecke der benannten Instanz k gezogen: Beschriftung „k.ecke“, beim Loslassen wird k.ecke geschrieben.

B04 Alt-Ziehen

WÄHREND



NACHHER



CODE VORHER

```
translate([5, 5, 2]) cube(4);
```

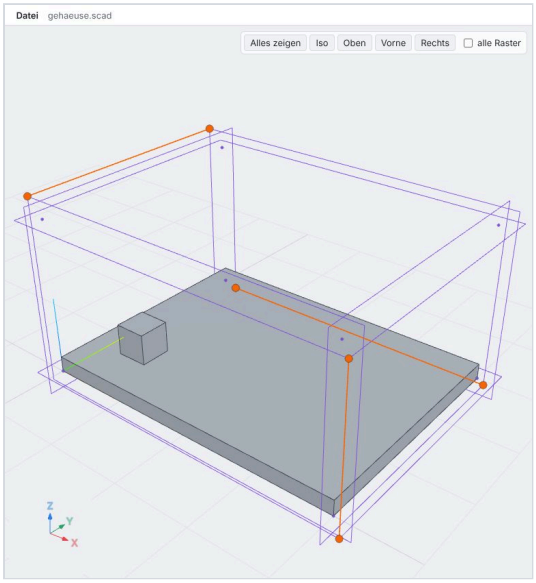
CODE NACHHER

```
translate([39, 31, 1]) cube(4);
translate([37, 28.5, -0.5]) cube(4);
```

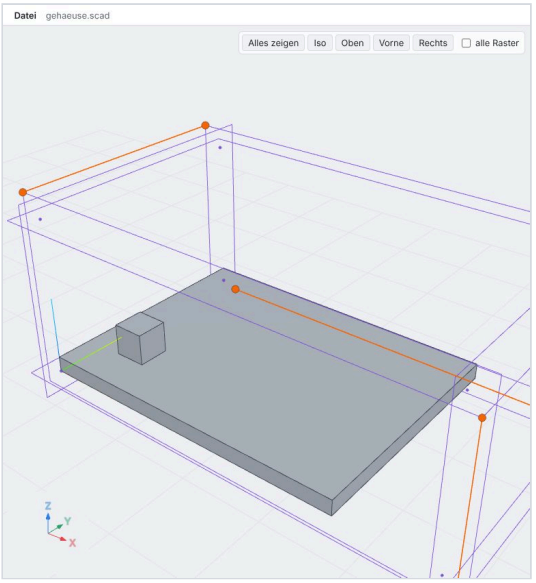
Mit gedrückter Alt-Taste nahe an einen Rasterpunkt gezogen: kein Snap, es werden Zahlen geschrieben, gerundet auf 1 mm (translate([39, 31, 1]) cube(4);). Danach Schrittweite in den Einstellungen auf 0,5 mm gesetzt: translate([37, 28.5, -0.5]) cube(4);.

B05 Rasterebene ziehen

VORHER



NACHHER



CODE VORHER

```
    x: links = 0, rechts = 40;
```

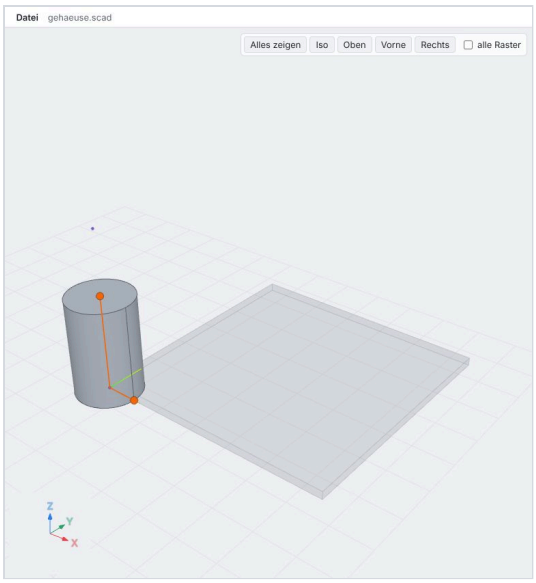
CODE NACHHER

```
    x: links = 0, rechts = 52;
```

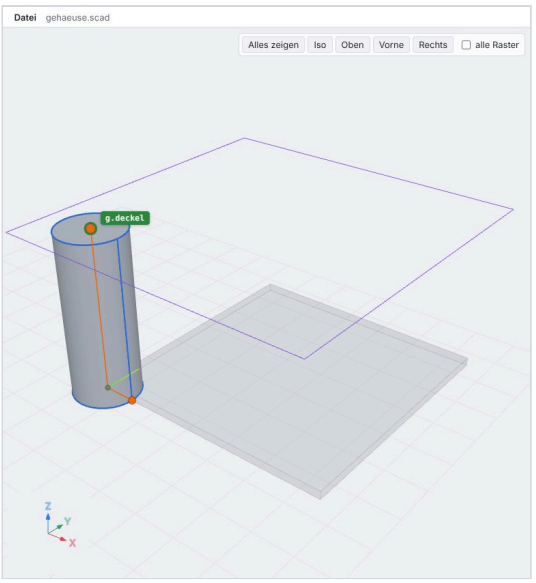
Cursor im Rasterblock: alle Ebenen mit Literal als durchgezogene Linie mit Griff. Der Griff der Ebene rechts wurde entlang der x-Achse gezogen; das Literal folgt, Linie und Rasterpunkte wandern mit.

B06 1D-Griff snappt auf Ebene

VORHER



WÄHREND



CODE VORHER

```
cylinder(h = 12, r = 4);
```

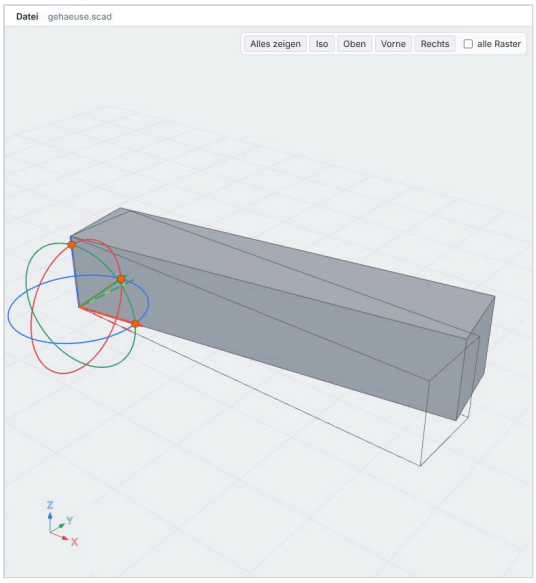
CODE NACHHER

```
cylinder(h = g.deckel, r = 4);
```

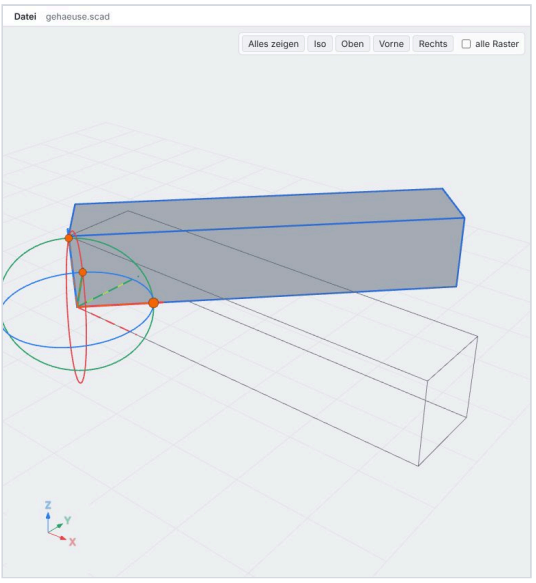
Höhengriff des Zylinders an die Rasterebene deckel gezogen: Ebenen derselben Achse sind Snap-Ziele, geschrieben wird g.deckel (neu in cad-eval: SnapRef::Plane).

B07 Drehring rastet in 15°

VORHER



NACHHER



CODE VORHER

```
rotate([0, 0, 10]) cube([30, 6, 6]);
```

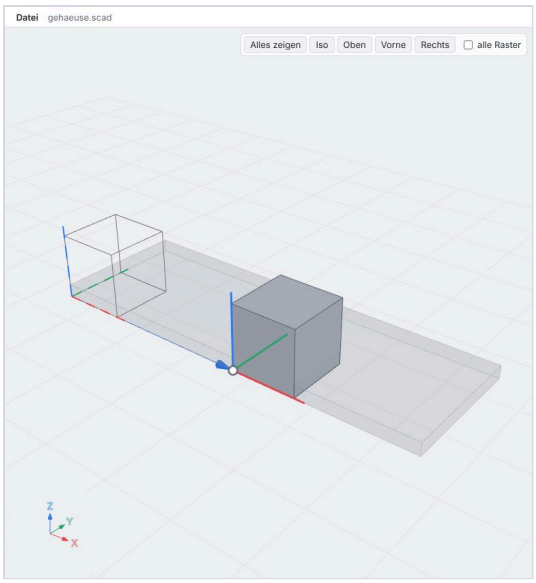
CODE NACHHER

```
rotate([0, 0, 45]) cube([30, 6, 6]);
```

Drehringe pro Achse; der Griff am z-Ring wurde von 10° bis 52° gezogen. Geschrieben werden nur Vielfache von 15° (Schrittweite für Winkel), am Ende 45°.

B08 Gesperrter Griff

VORHER



CODE VORHER

```
translate([a + 1, 0, 0] * 4) cube(5);
```

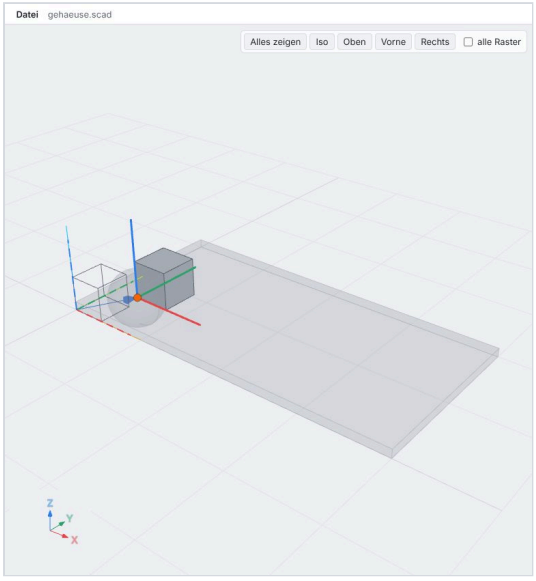
CODE NACHHER

```
translate([a + 1, 0, 0] * 4) cube(5);
```

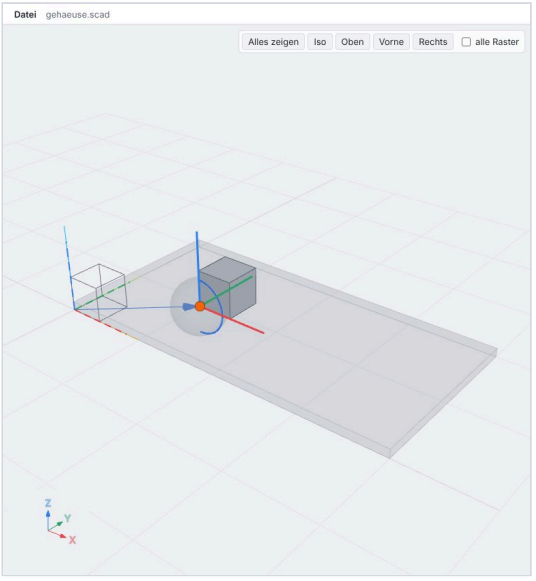
Berechneter Wert [a + 1, 0, 0] * 4: der Griff wird angezeigt (grauer Ring), ist aber gesperrt. Ziehen ändert nichts, die Statuszeile meldet „berechneter Wert – gesperrt“.

B09 Variable mit Literal

VORHER



NACHHER



CODE VORHER

```
p = [10, 0, 5];
translate(p) cube(4);
translate(p) sphere(3);
```

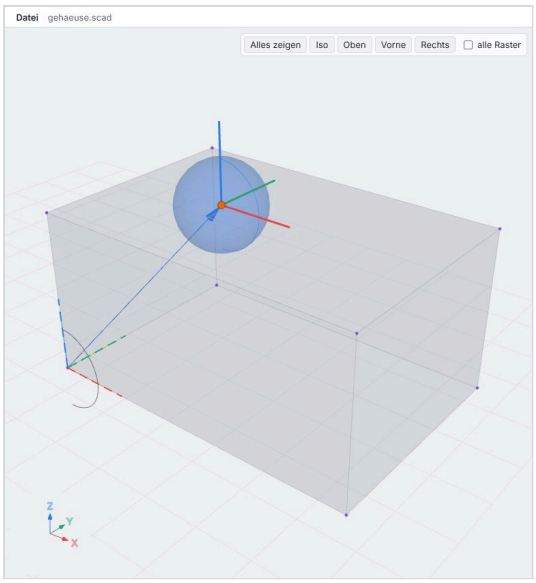
CODE NACHHER

```
p = [15, 4, 4];
translate(p) cube(4);
translate(p) sphere(3);
```

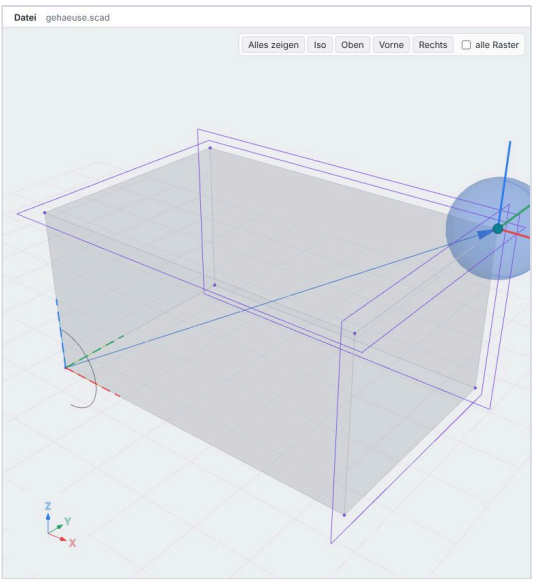
translate(p) in Zeile 2 gezogen: geändert wird die Definition von p in Zeile 1, Würfel und Kugel (Zeilen 2 und 3) verschieben sich um denselben Betrag.

B10 Zonenmittelpunkt

VORHER



NACHHER



CODE VORHER

```
    translate([10, 10, 15]) sphere(4);
```

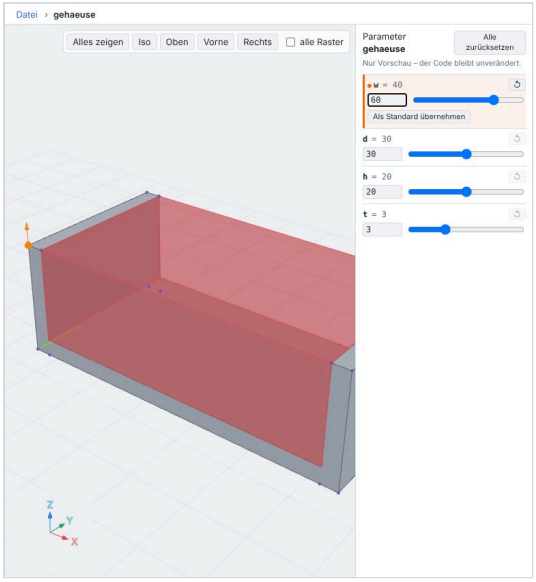
CODE NACHHER

```
    translate(g[rechts, hinten, deckel])
sphere(4);
```

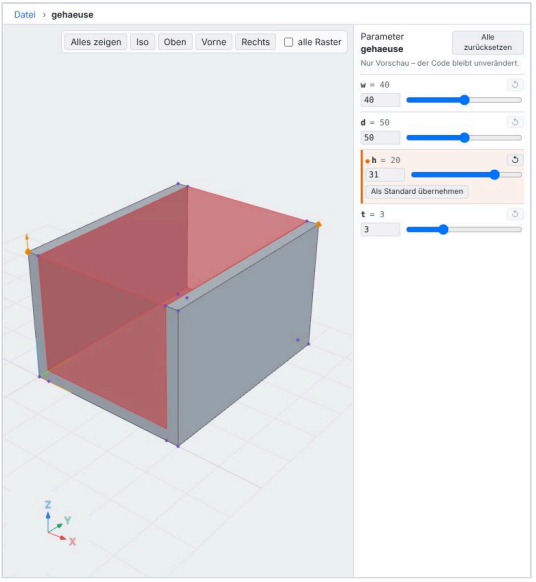
Cursor auf der Zone (Kugel) im fillet-Block: Griff am Zonenmittelpunkt (translate), auf die Ecke rechts/hinten/deckel gezogen. Die Zone wandert, die ausgewählten Kanten folgen.

B11 Parameterleiste

VORHER



NACHHER



CODE VORHER

```
module gehaeuse(w = 40, d = 30, h = 20, t = 3) {
```

CODE NACHHER

```
module gehaeuse(w = 40, d = 50, h = 20, t = 3) {
```

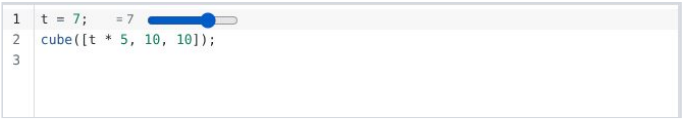
Cursor in module gehaeuse: Leiste mit w, d, h, t (Eingabefeld + Schieberegler, Reset ↺). w = 60 änderte nur die Vorschau (Code unverändert, Zeile markiert), Reset stellte 40 her; h per Schieberegler auf 31. d = 50 blieb beim Verlassen und Wiederbetreten des Moduls erhalten; „Als Standard übernehmen“ schrieb d = 50 in die Moduldefinition.

B12 Inline-Regler

VORHER



NACHHER



CODE VORHER

```
t = 3;
hohl = true;
```

CODE NACHHER

```
t = 7;
hohl = false;
```

Zahl-Literal einer Zuweisung: kleiner Schieberegler am Zeilenende, mit der Maus von 3 auf 7 gezogen (Code und Szene folgen live, ein Undo-Schritt). Bool-Literal: Schalter, Klick schreibt false; die Szene wechselt auf den else-Zweig.

B13 Sprung zur Rasterdeklaration

CODE VORHER

```
11: box(g[links, vorne, boden], g[rechts, hinten,
deckel]);
```

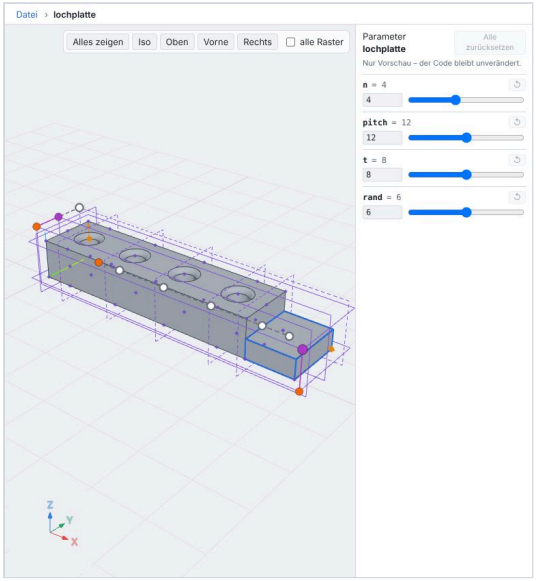
CODE NACHHER

```
4: grid g {   ← Cursor
```

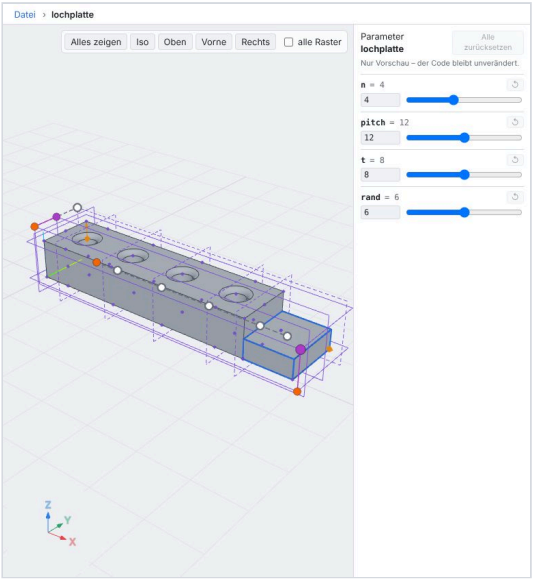
Strg+Klick auf g[links, vorne, boden] (Zeile 11) setzt den Cursor auf die Deklaration grid g (Zeile 4); ein normaler Klick setzt weiter nur den Cursor.

B14 Messwerte beim Ziehen

WÄHREND



NACHHER



CODE VORHER

```
module lochplatte(n = 4, pitch = 12, t = 4, rand =
6) {
```

CODE NACHHER

```
module lochplatte(n = 4, pitch = 12, t = 8, rand =
6) {
```

lochplatte (Differenz mit fillet = 1, vier Bohrungen): Ebene oben = t (Standardwert, andere Farbe) 78-mal bewegt, ein Schritt pro Frame. Ziehen → Szene grob: Median 371 ms, p95 738 ms (15 Szenen); Loslassen → feine Szene 202 ms; 21 fps während des Ziehens, 76 Drag-Nachrichten, 0 übersprungen.